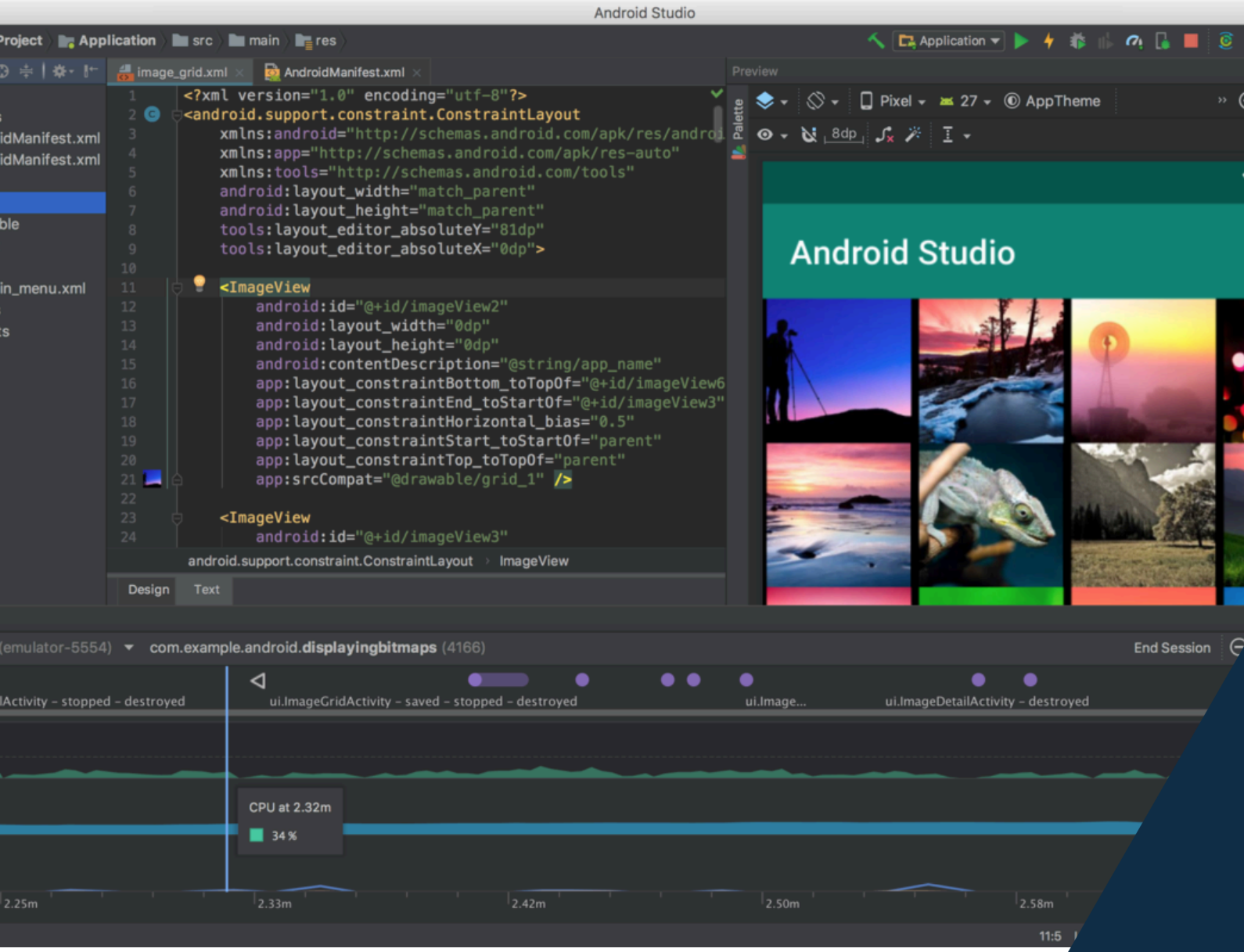


PROGRAMACIÓN MULTIMEDIA Y DISPOSITIVOS MÓVILES

Práctica 1 · Mi primera app Android (Temas 1-3)

David Alonso Aguilar

MEDAC
Instituto Oficial de Formación Profesional

Índice

1. Introducción 4
2. Parte A · Análisis del desarrollo móvil (Tema 1) 4
2.1. Ventajas e inconvenientes 4
2.2. Elección tecnológica 6
2.3. Ciclo de vida de una Activity 7
3. Parte B · Especificación de tu app (Temas 1-2) 12
3.1. Idea y público objetivo 12
3.2. Requisitos funcionales 13
3.3. Requisitos no funcionales 13
3.4. Casos de uso 14
4. Parte C · Entorno de desarrollo (Tema 2) 16
4.1. Crea el proyecto en Android Studio con el nombre PMDM_P1_TuNombreApellido y un paquete propio (es.medac.tunombre.app) 16
4.2. Configura dos dispositivos virtuales (AVD) distintos 17
4.3. Ejecuta la app en ambos y aporta capturas 18
4.4. Explica brevemente la estructura del proyecto 20
5. Parte D · Recursos de la aplicación (Tema 3) 21
5.1. Justificación de la paleta 21
5.2. Justificación técnica de formatos de imágenes 21
5.3. Capturas de la app en ambos idiomas 22
5.4. Modo oscuro 23
6. Conclusión 24
6.1. Qué he aprendido 24
6.2. Dificultades encontradas y soluciones 24

Índice de ilustraciones y tablas

Tabla 1: Elección tecnológica **Pág. 6**

Ilustración 1: Diagrama general del ciclo de vida del Activity **Pág. 8**

Ilustración 2: Diagrama de llamada del ciclo de vida del Activity **Pág. 9**

Ilustración 3: Diagrama giro de pantalla del ciclo de vida del Activity **Pág. 10**

Ilustración 4: Diagrama de botón atrás del ciclo de vida del Activity **Pág. 11**

Ilustración 5: Casos de uso **Pág. 14**

Ilustración 6: Captura de creación del proyecto **Pág. 16**

Ilustración 7: Captura de dispositivos AVD configurados **Pág. 17**

Tabla 2: Dispositivos AVD **Pág. 17**

Ilustración 8: Captura de la app ejecutada en Pixel 7 **Pág. 18**

Ilustración 9: Captura de la app ejecutada en Tablet **Pág. 19**

Ilustración 10: Captura de la app en inglés **Pág. 22**

Ilustración 11: Captura de la app en español **Pág. 22**

Ilustración 12: Captura de la app en modo oscuro **Pág. 23**

Práctica 1 · Mi primera app Android (Temas 1-3)

1. Introducción

El presente documento recoge la planificación, especificación, configuración del entorno y diseño de interfaz para PCbuilder, una aplicación móvil nativa desarrollada en Android Studio. El objetivo principal de la práctica consiste en asentar las bases del desarrollo móvil nativo mediante la comprensión del ciclo de vida de los componentes, la correcta estructuración del árbol de recursos (res/), el cumplimiento de buenas prácticas de diseño (cero hardcoding, uso estricto de dp/sp) y la adaptabilidad de la interfaz ante distintos dispositivos, densidades, idiomas y temas del sistema operativo.

2. Parte A · Análisis del desarrollo móvil (Tema 1)

2.1. Ventajas e inconvenientes

- **Ventajas**:
  - Portabilidad: Se utiliza directamente en la mesa de montaje o en la tienda física al comprar piezas.
  - Acceso a hardware: Permite usar funciones nativas como la cámara (para escanear componentes o códigos) y almacenamiento local.
  - Uso sin conexión: Funciona de forma autónoma sin depender de un navegador ni de conexión a internet.
- **Desventajas**:
  - Hardware limitado: Menor memoria RAM y restricciones térmicas/batería frente a un ordenador.
  - Alta fragmentación: Diversidad de tamaños de pantalla, densidades y versiones de Android.
  - Ciclo de vida restrictivo: El sistema operativo puede pausar o cerrar la app en cualquier momento para liberar recursos.

MEDAC

Cuatro limitaciones y su impacto en PCbuilder:

- Memoria RAM
    - Impacto: Si el usuario cambia a YouTube para ver cómo instalar el disipador, Android puede destruir la Activity de PCbuilder en segundo plano.
    - Solución: Gestionar el ciclo de vida para no perder datos introducidos.
- Tamaño de pantalla
    - Impacto: No se pueden mostrar tablas densas con decenas de especificaciones técnicas a la vez.
    - Solución: Diseñar tarjetas resumidas y usar listas con scroll (ConstraintLayout).
- Batería
    - Impacto: El ensamblaje de un PC puede durar horas, una app pesada agotaría el móvil.
    - Solución: Evitar procesos en segundo plano cuando la app no esté visible.
- Fragmentación de pantallas
    - Impacto: Distintos dispositivos y resoluciones podrían descolocar botones o hacerlos ilegibles.
    - Solución: Usar exclusivamente dp para márgenes y tamaños, y sp para textos.

2.2. Elección tecnológica

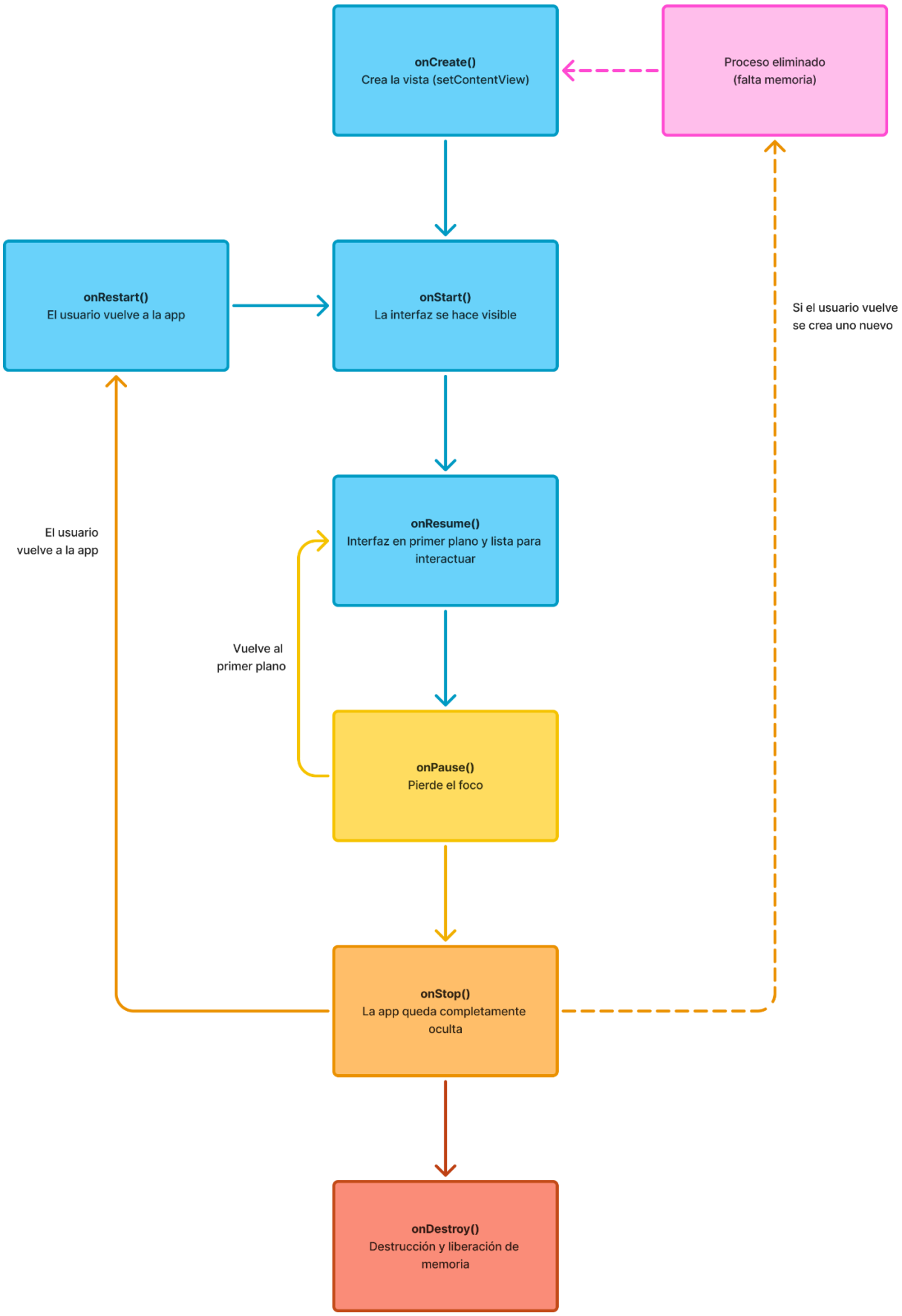
Criterio	Nativo	Híbrido	Multiplataforma
Lenguajes	Java / Kotlin	HTML5, CSS, JavaScript	Dart (Flutter), JS/TS (React Native)
Rendimiento	Máximo (código compilado directo para la máquina/SO)	Bajo (se ejecuta dentro de un navegador web o WebView)	Alto (renderizado propio o puente a componentes nativos).
Acceso a hardware	Total e inmediato a través de la API oficial de Android	Limitado (depende de plugins de terceros)	Bueno (mediante librerías de la comunidad o módulos puente).
Coste	Mayor si se requiere versión para iOS (dos códigos fuente)	Mínimo (un único código web para todas las plataformas)	Medio (una sola base de código para Android e iOS).
Ejemplo de framework	Android SDK (Android Studio)	Apache Cordova / Ionic clásico	Flutter / React Native

Mi app se desarrollará en nativo android por los siguientes puntos:

- **Rendimiento y fluidez**: Garantiza una tasa de refresco constante (60/120 fps) al navegar por listas de componentes y formularios sin el retraso típico de las vistas web embebidas.
- **Control del ciclo de vida**: Permite gestionar con precisión el estado de la actividad para no perder datos introducidos cuando el sistema libera memoria.
- **Base pedagógica**: Permite comprender la arquitectura real de Android (vistas XML, sistema de recursos res/ y ciclo de vida de componentes) antes de utilizar capas de abstracción multiplataforma.

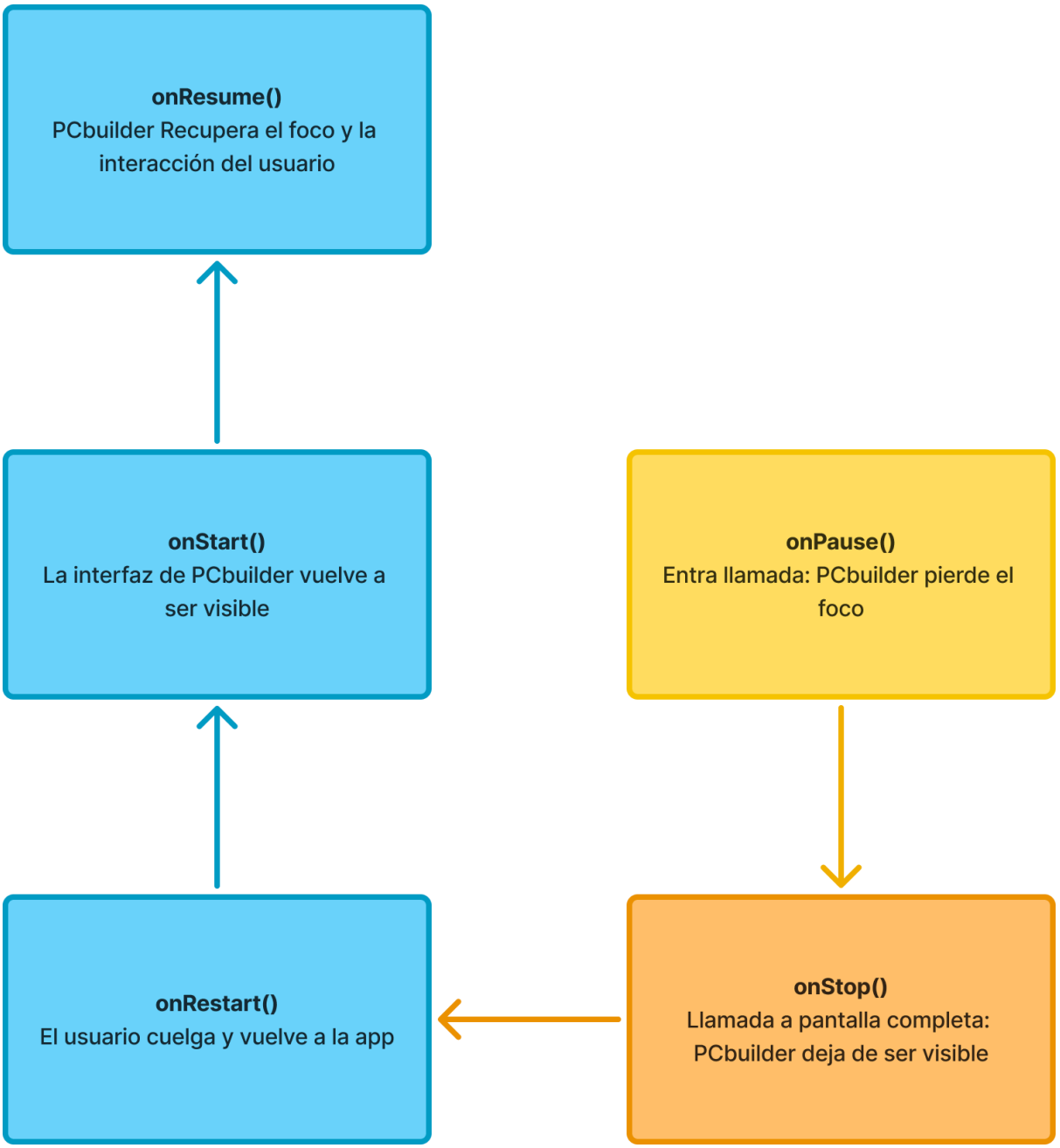
2.3. Ciclo de vida de una Activity

- **Diagrama general**: Lo pongo en la siguiente página para que se vea la imágen completa.

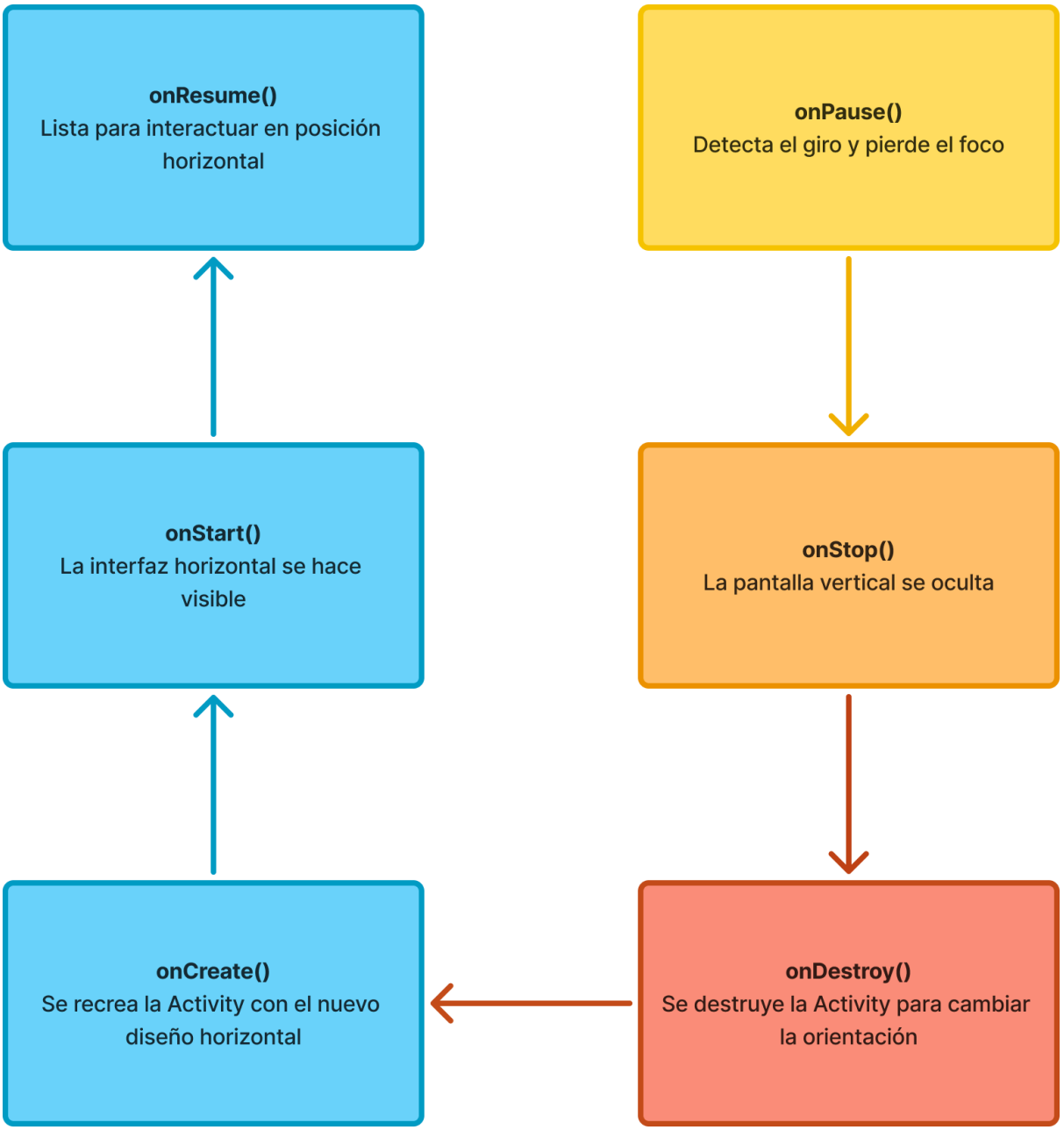

onCreate()
Crea la vista (setContentView)
Proceso eliminado
(falta memoria)
onRestart()
El usuario vuelve a la app
onStart()
La interfaz se hace visible
Si el usuario vuelve
se crea uno nuevo
El usuario
vuelve a la app
onResume()
Interfaz en primer plano y lista para
interactuar
Vuelve al
primer plano
onPause()
Pierde el foco
onStop()
La app queda completamente
oculta
onDestroy()
Destrucción y liberación de
memoria

MEDAC

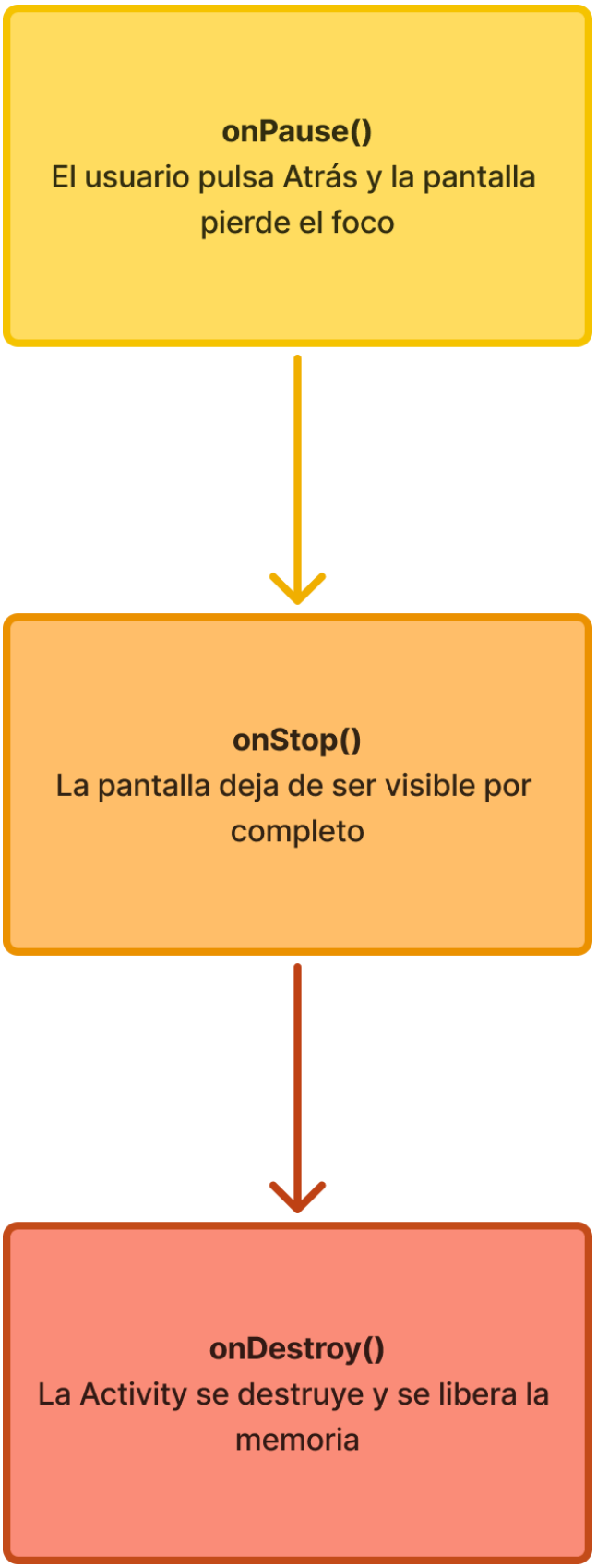
- **Diagrama de llamada**:



- **Diagrama de giro de pantalla**:



- **Diagrama de botón atrás**:



3. Parte B · Especificación de tu app (Temas 1-2)

3.1. Idea y público objetivo

PCbuilder es una aplicación móvil nativa para Android concebida como un asistente técnico y de control presupuestario para el montaje de ordenadores personalizados. Permite registrar componentes estructurados por categorías (procesador, tarjeta gráfica, placa base...), hacer seguimiento del estado de adquisición de cada pieza y calcular el coste total del equipo en tiempo real. La herramienta está diseñada para operar íntegramente sin conexión, facilitando su consulta directa en tiendas físicas o sobre el banco de trabajo durante el ensamblaje. Su público objetivo está compuesto por técnicos de montaje y soporte, estudiantes del sector informático y entusiastas del hardware que necesitan una solución ágil, visual y estructurada frente a las hojas de cálculo tradicionales o anotaciones dispersas.

Objetivos:

- **Agilidad de entrada de datos**: Permitir al usuario registrar un nuevo componente, asignándole categoría, modelo, precio y estado, en menos de 15 segundos mediante campos guiados y selectores directos.
- **Rendimiento y consumo de recursos**: Lograr un tiempo de inicio de la app inferior a 1,5 segundos y mantener el consumo de memoria RAM por debajo de 60 MB durante la navegación entre pantallas.
- **Disponibilidad y persistencia offline**: Garantizar el 100% de disponibilidad y funcionamiento sin conexión a internet, almacenando y recuperando todas las configuraciones de forma instantánea en el almacenamiento local del dispositivo.

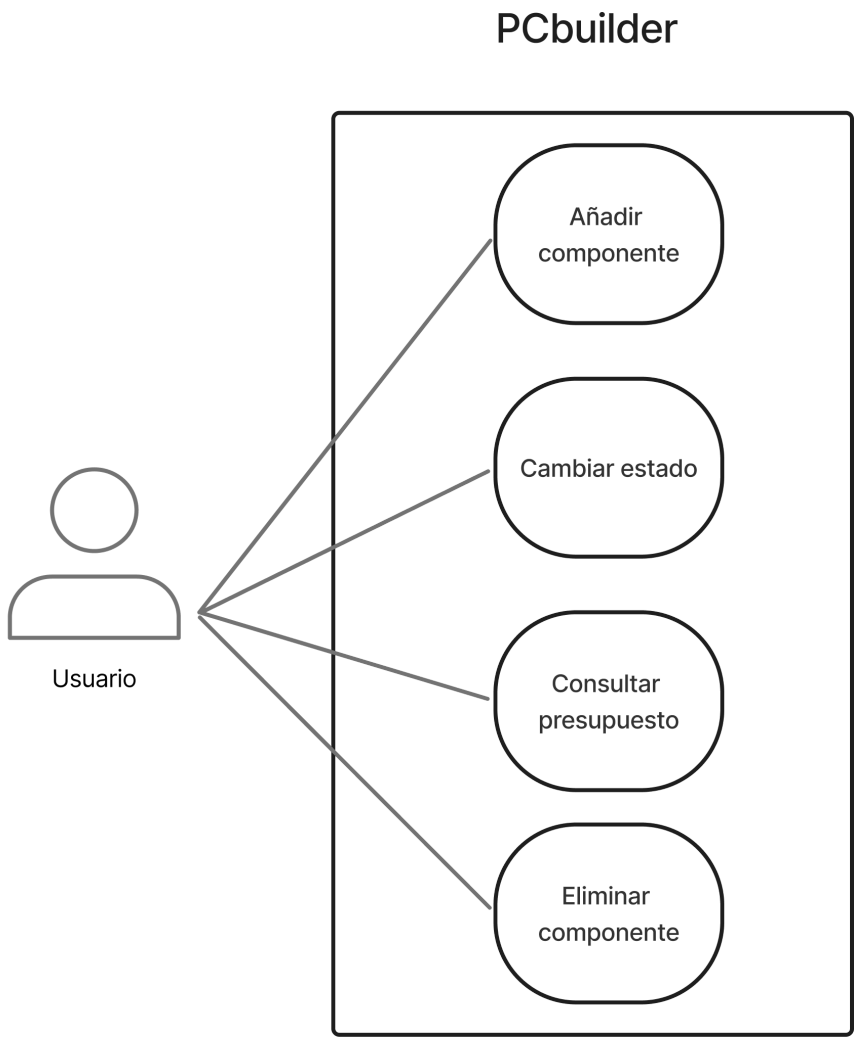
3.2. Requisitos funcionales

- RF-01 **Añadir componente** (Prioridad Alta): El usuario puede registrar una pieza indicando nombre/modelo, categoría (CPU, GPU, RAM, etc.) y precio estimado o real.
- RF-02 **Gestión de estado** (Prioridad Alta): Cada componente se puede marcar con un estado: Pendiente, Comprado o Montado.
- RF-03 **Cálculo del presupuesto** (Prioridad Alta): La app calcula y muestra en tiempo real el coste total del equipo, desglosando lo ya gastado frente a lo pendiente de compra.
- RF-04 **Edición y borrado** (Prioridad Media): Permite modificar los datos de cualquier pieza guardada o eliminarla de la lista si se cambia de idea.
- RF-05 **Filtrado por categoría** (Prioridad Baja): El usuario puede filtrar la lista para ver solo piezas de un tipo concreto o ver el montaje completo.

3.3. Requisitos no funcionales

- RNF-01 (**Compatibilidad**): La versión mínima compatible será Android 7.0 (API 24). Permite utilizar características modernas de Java (como expresiones lambda y la API Streams) garantizando compatibilidad con más del 99% de los dispositivos Android en uso sin necesidad de librerías adicionales de retrocompatibilidad.
- RNF-02 (**Rendimiento**): El tiempo de inicio de la pantalla principal debe ser inferior a 1,5 segundos, y la transición entre pantallas debe ejecutarse a 60 fps estables sin bloqueos en el hilo principal (Main Thread).
- RNF-03 (**Usabilidad**): Todos los elementos interactivos (botones, selectores y campos de texto) deben tener un área táctil mínima de 48 × 48 dp, garantizando pulsaciones cómodas y accesibles con una sola mano según las guías de Material Design.
- RNF-04 (**Mantenibilidad y localización**): El 100% de los textos, dimensiones y colores deben estar externalizados en res/values/ (strings.xml, dimens.xml, colors.xml), con cero valores introducidos directamente a mano en el código Java o en los layouts XML.

3.4. Casos de uso



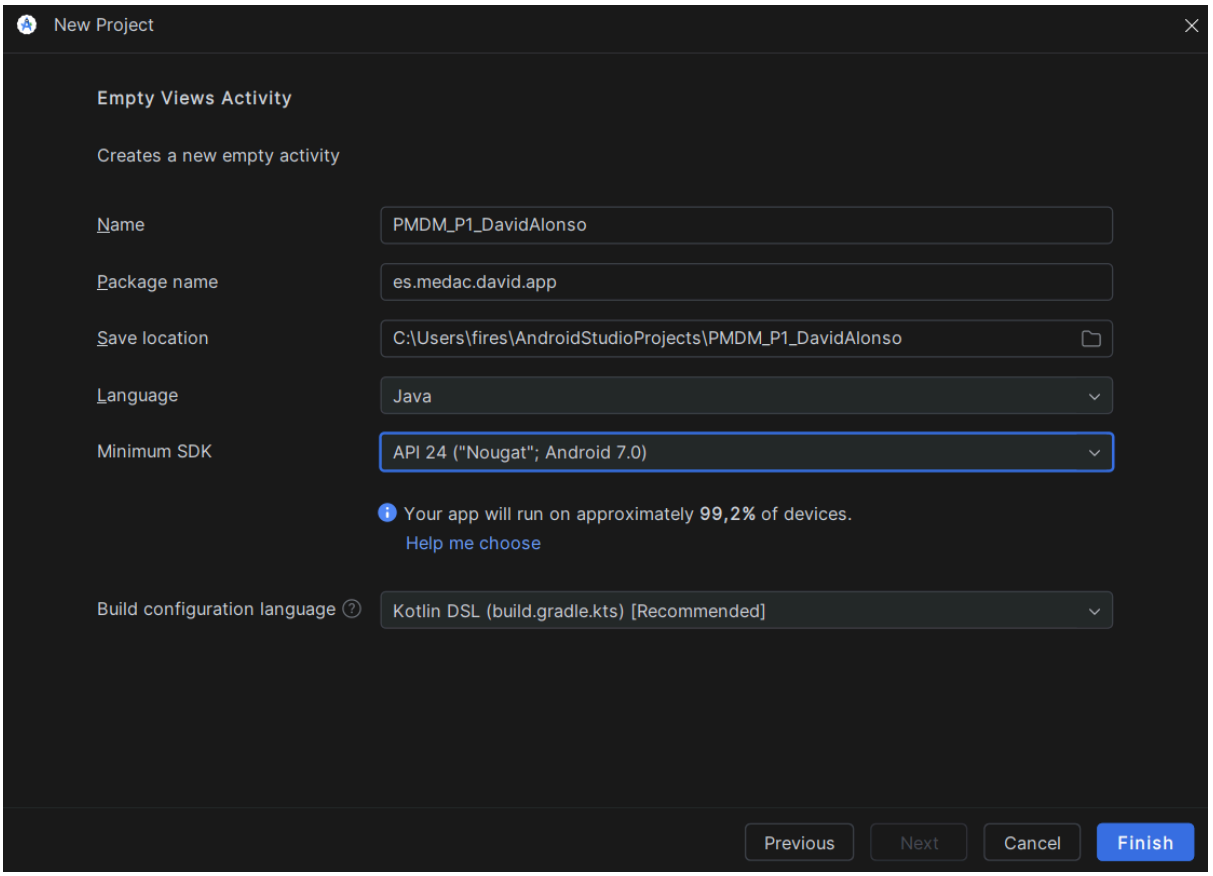
- Ficha detallada de Cambiar estado del montaje:
    - Actor: Usuario.
    - Precondición: Existe al menos un componente añadido al montaje.
    - Flujo principal:
        - El usuario localiza la pieza en la lista y pulsa sobre su estado actual (ej. Pendiente).
        - El sistema despliega las opciones disponibles (Pendiente, Comprado, Montado).
        - El usuario selecciona el nuevo estado (ej. Comprado).
        - El sistema actualiza el registro local y cambia el color/etiqueta de la tarjeta.

MEDAC

    - Flujo alternativo (Cancelar cambio): El usuario pulsa fuera del selector sin elegir una opción, el sistema mantiene el estado anterior sin cambios.
    - Postcondición: La pieza refleja su nuevo estado y se recalcula el importe de lo ya gastado.
- Ficha detallada de Añadir componente:
    - Actor: Usuario.
    - Precondición: La app está abierta en la pantalla principal.
    - Flujo principal:
        - El usuario pulsa el botón "Añadir pieza".
        - El sistema muestra el formulario con campos de datos y selector de categoría.
        - El usuario elige la categoría (ej. Procesador), escribe el modelo y el precio estimado.
        - El usuario pulsa "Guardar".
        - El sistema valida los datos, registra la pieza en la base de datos local y actualiza la lista.
    - Flujo alternativo (Datos incompletos): Si falta el nombre o el precio no es válido, el sistema resalta el campo con error y no guarda hasta que se corrija.
    - Postcondición: El nuevo componente aparece en el listado y suma su coste al presupuesto total.

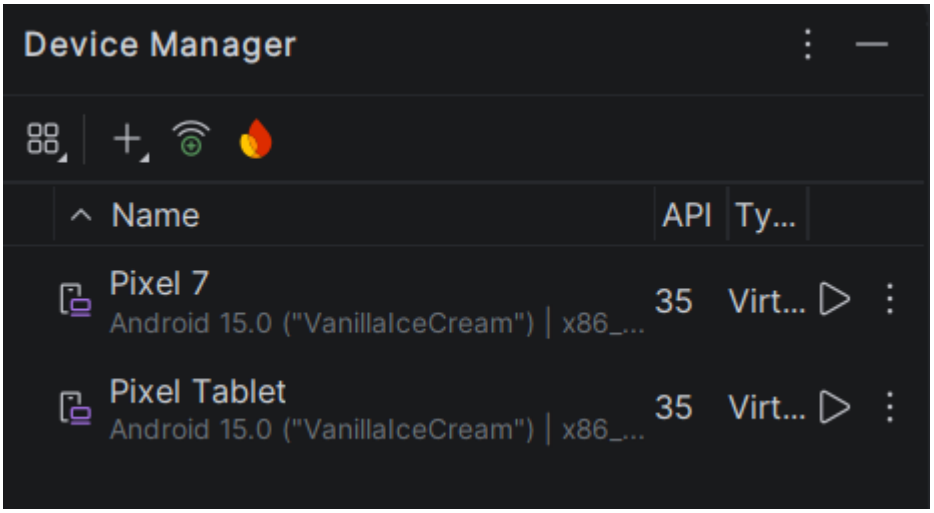
4. Parte C · Entorno de desarrollo (Tema 2)

4.1. Crea el proyecto en Android Studio con el nombre PMDM_P1_TuNombreApellido y un paquete propio (es.medac.tunombre.app)



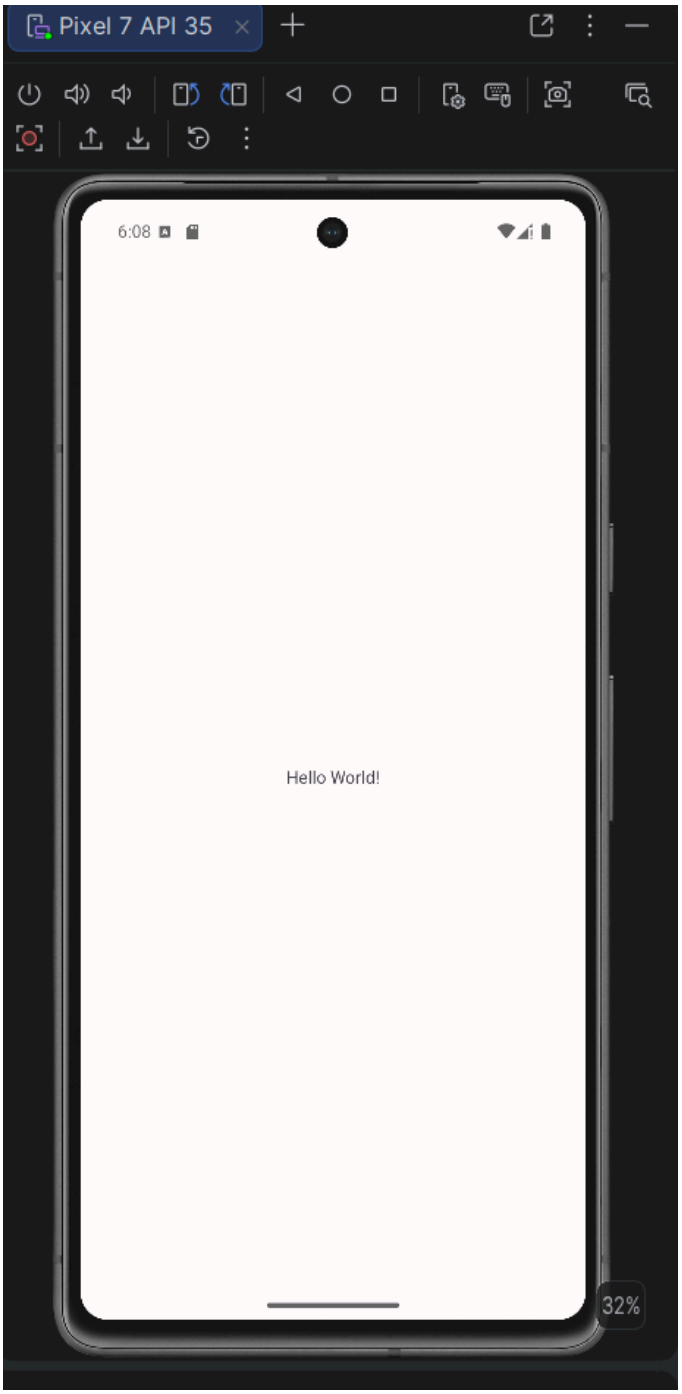
En Build configuration language he seleccionado Kotlin DSL porque en las nuevas versiones de Android Studio es la única opción disponible.

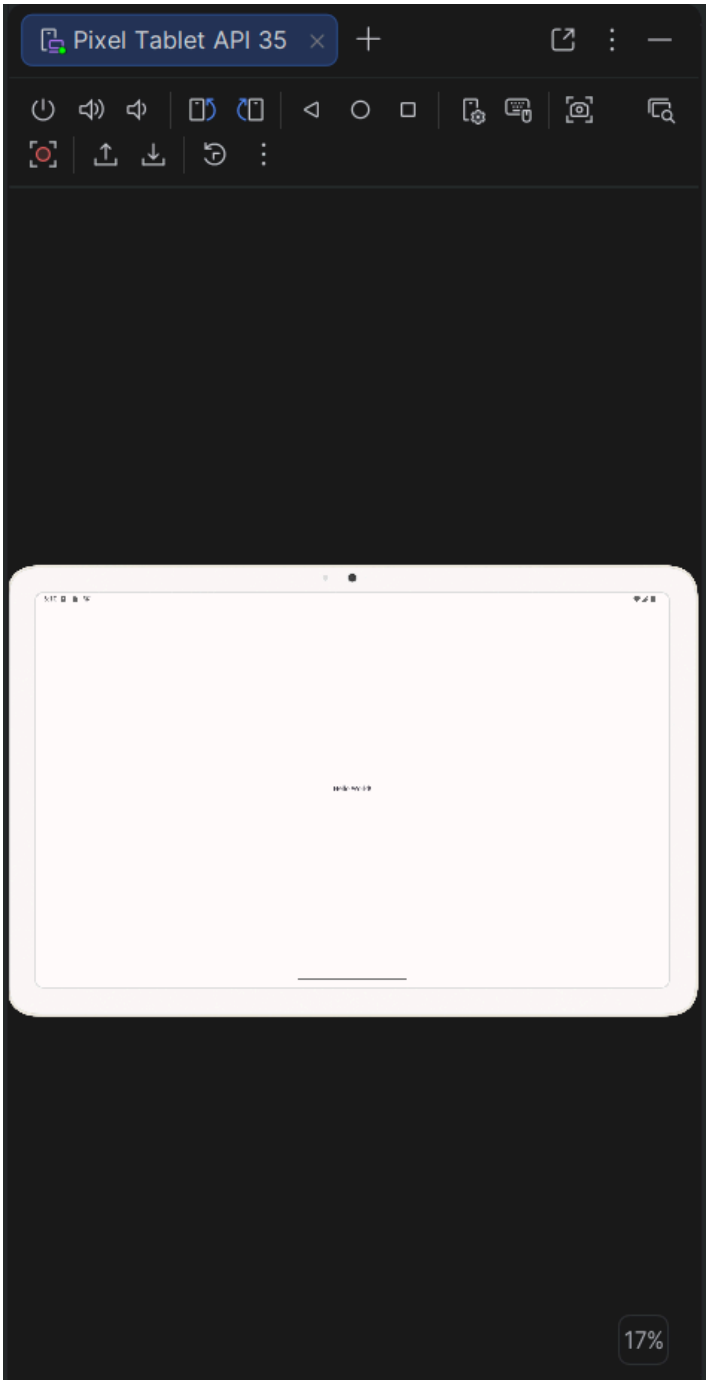
4.2. Configura dos dispositivos virtuales (AVD) distintos



Dispositivo virtual	Tipo de pantalla	Resolución	Densidad (dpi)	Versión SO (API)
Pixel 7	Teléfono inteligente	1080 x 2400 px	420 dpi (xxhdpi)	Android 15 (API 35)
Tablet	Tableta apaisada	2560 x 1600 px	320 dpi (xhdpi)	Android 15 (API 35)

4.3. Ejecuta la app en ambos y aporta capturas




Pixel Tablet API 35
17%

4.4. Explica brevemente la estructura del proyecto

- AndroidManifest.xml: Fichero de configuración global de la aplicación. Declara el paquete, permisos del sistema, icono, tema y registra los componentes principales (Activities), definiendo cuál es la pantalla de inicio (launcher).
- java/: Contiene el código fuente en Java (MainActivity.java). Aquí reside la lógica de negocio, control de eventos y manipulación de datos.
- res/ (Resources): Almacena los recursos estáticos externos al código para evitar el hardcoding:
    - layout/: Vistas y diseño de interfaces en XML.
    - values/: Constantes (strings.xml para textos, colors.xml para la paleta de colores y dimens.xml para medidas y márgenes).
    - drawable/ y mipmap/: Gráficos vectoriales e iconos de la aplicación.
- Ficheros Gradle (build.gradle.kts): Scripts de compilación y gestión de dependencias del proyecto.
    - Nivel proyecto: Configuración y plugins globales.
    - Nivel módulo (app): Define versiones de SDK y librerías externas.
- Relación entre la Activity (Java) y su Layout (XML): Sigue el principio de separación entre presentación y lógica:
    - Layout XML (activity_main.xml): Define de forma visual y declarativa la interfaz de usuario.
    - Activity Java (MainActivity.java): Controla el comportamiento y el ciclo de vida de la pantalla.
    - Vinculación: Dentro del método onCreate(), la Activity carga e infla la interfaz mediante setContentView(R.layout.activity_main);. A partir de ahí, la clase Java interactúa con las vistas mediante sus identificadores únicos (id), manteniendo el código desacoplado del diseño.

5. Parte D · Recursos de la aplicación (Tema 3)

5.1. Justificación de la paleta

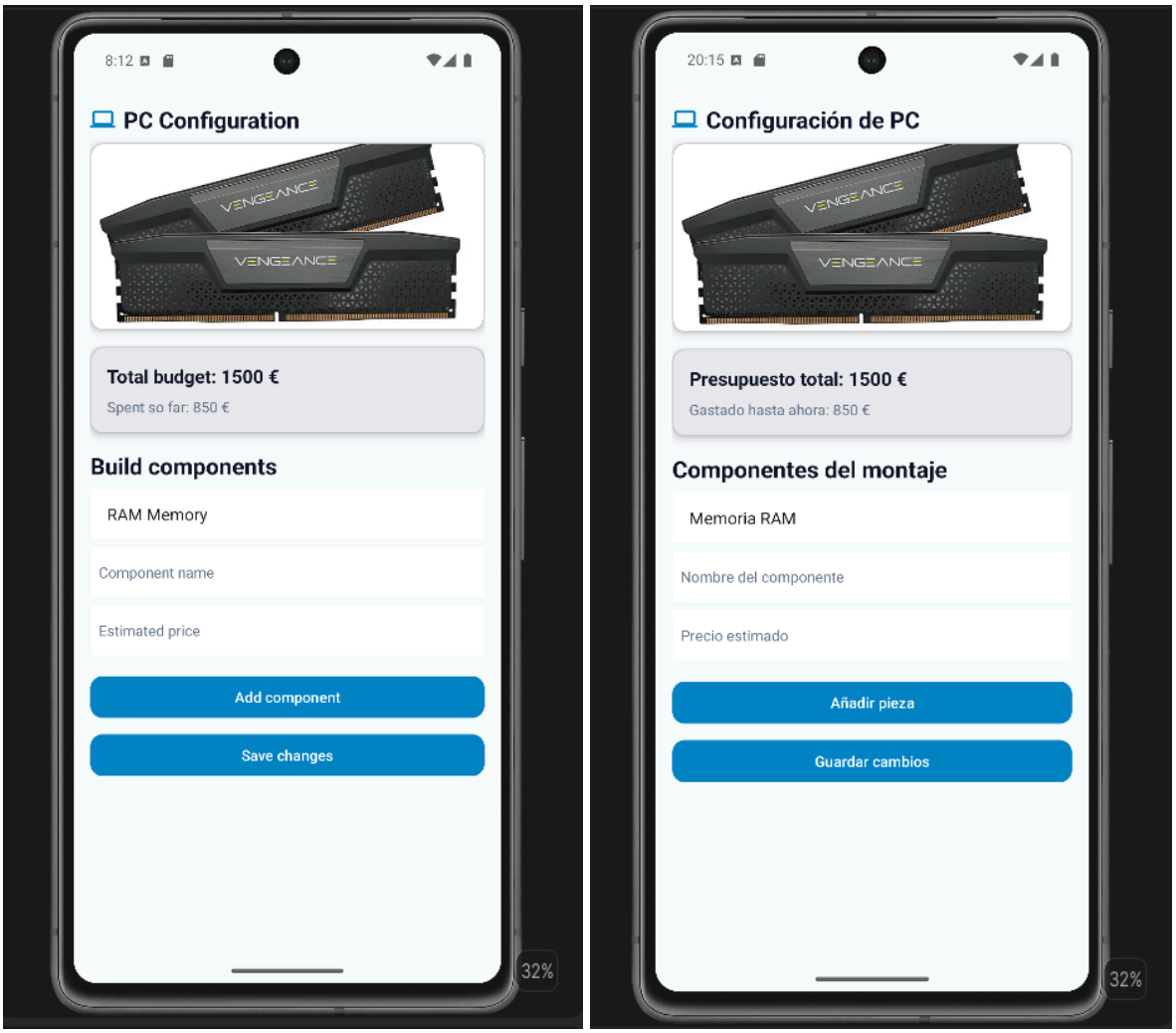
- **Temática tecnológica**: Se utiliza una base sobria y profesional de tonos pizarra y acero (color_primario y color_fondo), habitual en software de diagnóstico y configuración de hardware.
- **Nombres funcionales**: Ningún color se nombra por su valor cromático (no hay "azul" ni "gris"), sino por el rol que desempeña en la interfaz (color_superficie, color_secundario, etc.).
- **Contraste y usabilidad**: El texto principal (#0F172A) sobre la superficie blanca (#FFFFFF) y el fondo claro (#F8FAFC) cumple holgadamente los estándares de contraste y accesibilidad.
- **Semántica de estados**: Se reservan colores específicos para el seguimiento del montaje (color_estado_comprado en verde y color_estado_pendiente en naranja) para dar retroalimentación visual inmediata.

5.2. Justificación técnica de formatos de imágenes

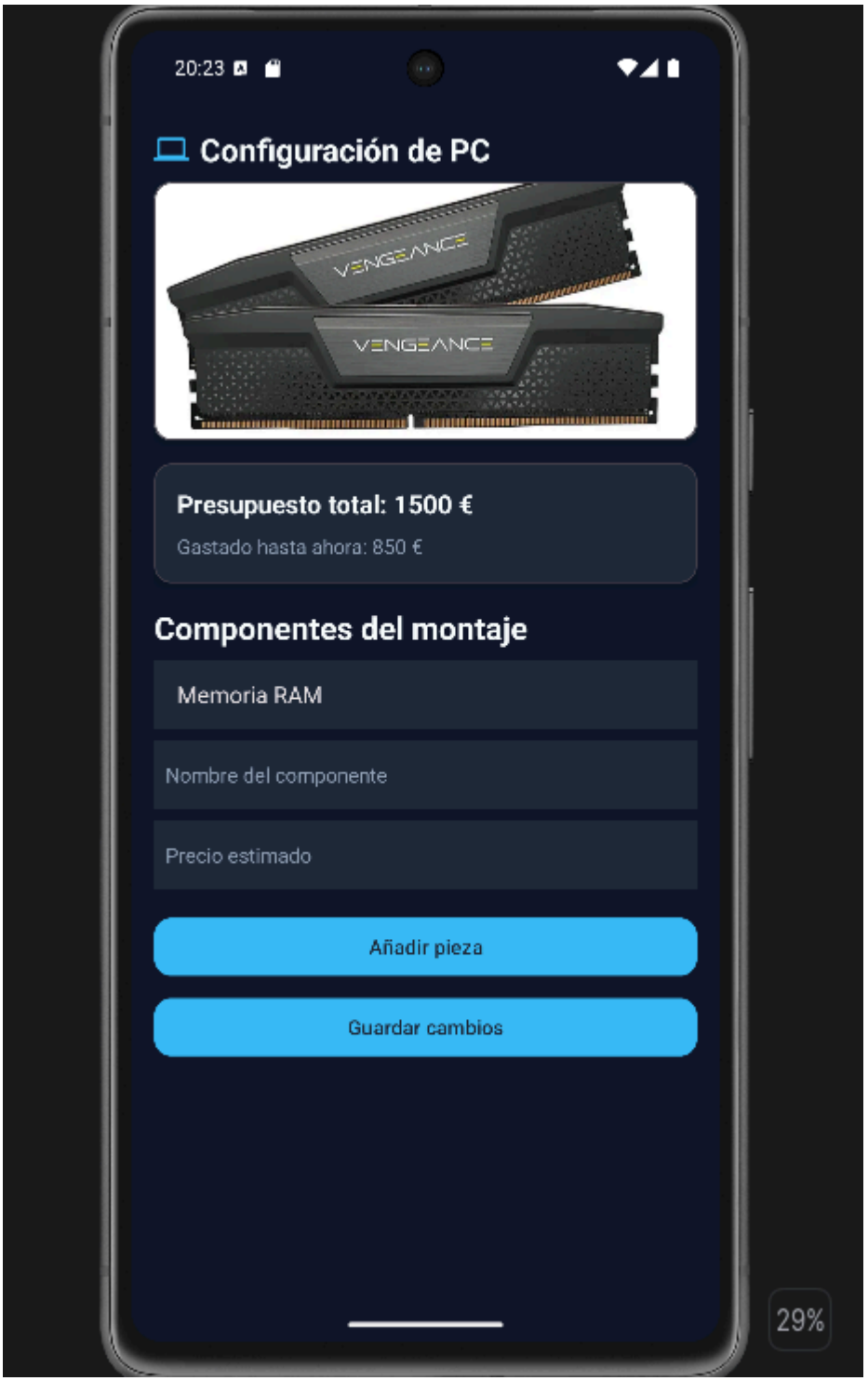
- **Formato vectorial** (Vector Asset / XML):
    - Uso recomendado: Iconos de interfaz, glifos, botones y elementos geométricos sencillos.
    - Impacto en el tamaño: Al definirse mediante coordenadas matemáticas y comandos de dibujo (path), un solo archivo XML sirve para todas las resoluciones de pantalla (desde mdpi hasta xxxhdpi) sin perder calidad ni pixelarse. Ocupa unos pocos kilobytes, minimizando drásticamente el peso del APK/AAB y evitando duplicar archivos por densidad.
- **Mapa de bits** (PNG / WebP / JPEG):
    - Uso recomendado: Fotografías realistas de productos, renders con texturas complejas o capturas detalladas que no pueden traducirse eficientemente a fórmulas vectoriales sin sobrecargar la CPU al renderizar.

- Impacto en el tamaño: Cada píxel almacena información de color. Para que no se pixelen en pantallas de alta densidad, la plataforma exige exportar la imagen en múltiples carpetas de densidad (drawable-mdpi, drawable-hdpi, drawable-xhdpi, etc.), multiplicando de forma considerable el tamaño final de la aplicación.

5.3. Capturas de la app en ambos idiomas



5.4. Modo oscuro



6. Conclusión

6.1. Qué he aprendido

A lo largo de esta práctica he comprendido en profundidad el flujo de trabajo del ecosistema nativo de Android. Especialmente valioso ha sido entender el desacoplamiento real entre la lógica de negocio en Java y la capa visual declarativa en XML mediante el inflado de vistas en el método onCreate(). Asimismo, he asimilado la importancia de gestionar los recursos externos (strings.xml, colors.xml, dimens.xml) para garantizar mantenibilidad, accesibilidad y una internacionalización limpia y desacoplada mediante calificadores de recursos (values-en, values-night).

6.2. Dificultades encontradas y soluciones

- Al sustituir el layout base por un contenedor ScrollView con scroll, la configuración por defecto de Android Studio generó un error de resolución de identificadores al buscar el ID raíz main. Lo arreglé asignando dicho identificador directamente al contenedor padre en el XML para preservar la compatibilidad con barras del sistema.
- Al emplear el editor visual de traducciones (Translations Editor), los elementos de tipo string-array destinados al selector desplegable no se trasladaban de forma automatizada al fichero inglés, provocando que el desplegable se mostrara en español mientras el resto de la interfaz estaba en inglés. Lo resolví declarando el bloque string-array manualmente dentro de res/values-en/strings.xml.
- Al configurar la variante de modo oscuro en values-night, un error de duplicidad en el prólogo XML impidió la compilación inicial por una excepción SAX, solucionándose al sanear la estructura del fichero y asegurar la cabecera única en la primera línea.